Assignment-4
Parallel and Distributed Computing

Submitted by:

Shreyash Tripathi (102303328)

Q1: Ring Communication

Objective: Write an MPI program where each process sends a message to the next process in a ring topology.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup	Efficiency	Comm %
1	< 0.000001	N/A	N/A	0.00%
2	0.000009	~0.00	~0.00%	100.00%
4	0.000013	~0.00	~0.00%	100.00%

Execution Output:

Process 0 started with value: 100
Process 1 received value: 100
Process 2 received value: 101
Process 3 received value: 104
Process 0 received final wrapped value: 107

Analysis & Key Inferences:

- **Inference 1: Pure Latency Scaling.** The P=1 time evaluates to 0.000000 seconds because local arithmetic takes fractions of a microsecond. Therefore, the execution times for P=2 and P=4 represent pure MPI communication latency. The time nearly doubles from P=2 (0.000009s) to P=4 (0.000013s). This proves that ring topologies force strict O(p) sequential communication serialization.
- **Inference 2: Overhead Dominance.** Because the arithmetic payload as of by adding the rank is infinitesimally small, the communication overhead is 100%. The system spends its entire execution time routing MPI_Send and MPI_Recv packets through the middleware.

- **System Design Takeaway:** Ring communication provides excellent ordered synchronization but zero computational acceleration. It should be used for token-passing or bounds-checking, not for speeding up data processing.

Q2: Parallel Array Sum

Objective: Compute the sum of an array of size 100 in parallel using MPI_Scatter and MPI_Reduce.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup (Sp)	Efficiency (Ep)	Comm %
1	0.000016	1.00	100.00%	0.00%
2	0.000085	0.2	9.50%	90.43%
4	0.000075	0.22	5.25%	95.86%

Execution Output:

Global Sum: 5050 (Expected: 5050)
Average Value: 50.50

Analysis & Key Inferences:

- **Inference 1: The Parallelization Threshold.** For an array of just 100 elements, calculating the sum sequentially on a single core takes a microscopic 0.000016 seconds. Invoking the MPI stack to scatter chunks of 25 or 50 integers actively harms execution time, slowing the program down significantly.
- **Inference 2: OS Scheduling Variance.** Interestingly, P=4 (0.000075 s) was slightly faster than P=2 (0.000085 s). At this micro-scale, MPI execution is heavily influenced by operating system thread scheduling and cache distribution. However, communication overhead still dominates at approx 96%.
- **System Design Takeaway:** To achieve a Speedup > 1 , the array size must be massively scaled (e.g N = 10^7) to ensure the math phase is long enough to mask the latency penalty of MPI_Scatter and MPI_Reduce.

Q3: Finding Maximum and Minimum

Objective: Generate 10 random numbers per process and find the global maximum and minimum values using MPI_MAXLOC and MPI_MINLOC.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup (Sp)	Efficiency (Ep)	Comm %
1	0.000005	1.00	100.00%	0.00%
2	0.000031	0.15	7.50%	92.42%
4	0.000017	0.31	7.75%	92.19%

Execution Output:

Process 0 generated: 897 562 661 317 604 148 935 477 177 905
Process 1 generated: 215 24 343 559 774 486 720 151 126 890
Process 2 generated: 382 991 223 209 16 206 946 625 650 527
Process 3 generated: 885 201 775 786 493 220 764 136 328 468
--- Global Results —
Global Maximum: 991 (Found in Process 2)
Global Minimum: 16 (Found in Process 2)

Analysis & Key Inferences:

- **Inference 1: Struct Transmission Cost.** Utilizing MPI_2INT to transmit custom val_rank structs for location tracking slightly increases the packet payload compared to a standard scalar reduction. This contributes to a massive >92% communication overhead.
- **Inference 2: Multi-core Cache Efficiency.** The execution time halved from P=2 (0.000031 s) to P=4 (0.000017 s). Generating random numbers triggers CPU interrupts. Dividing those interrupts across 4 physical cores allowed the hardware to execute the random generation phase fast enough to overcome a portion of the MPI reduction latency.

Q4: Parallel Dot Product

Objective: Compute the dot product of two vectors of size 8 in parallel.

Performance Data:

Processes (p)	Time (Tp) in sec	Speedup (Sp)	Efficiency (Ep)	Comm %
1	0.000013	1.00	100.00%	0.00%
2	0.000040	0.33	16.50%	83.75%
4	0.000087	0.15	3.75%	96.26%

Execution Output:

Parallel Dot Product Result: 120
Expected Result: 120

Analysis & Key Inferences:

- **Inference 1: Extreme Fine-Grained Imbalance.** Vector A and Vector B are incredibly small (N=8). At 4 processes, each node is responsible for computing exactly two multiplications before hitting the MPI_Reduce barrier.
- **Inference 2: Negative Scaling Triggered.** As we expanded from 2 processes to 4 processes, the execution time spiked from 0.000040 s to 0.000087 s, and efficiency crashed to 3.75% . The setup time required to scatter the arrays and gather the result acts as a sequential bottleneck.
- **System Design Takeaway:** Distributing workloads across a cluster is fundamentally inefficient when the computational phase is practically non-existent. The network overhead actively throttles the system. A single processor will always outperform MPI on microscopic datasets.